多AGENT架构

从入门到精通

程佳路

2026-04-21

多Agent架构

作者： 程佳路

出版： 探微观智

版本： v1.0

日期： 2026-04-21

简介： 从单Agent到多Agent协作的完整实战指南。10章 · 3 Part · 覆盖LangGraph/CrewAI/OpenAI SDK/AutoGen/Google ADK + MCP/A2A协议栈。

版权所有 © 2026 程佳路。保留所有权利。
未经书面许可，不得以任何形式复制或传播本书内容。

目录

Part 1: 起步

§ 01 单Agent到头了，然后呢

§ 02 架构模式和任务拆分：选错了全白干

§ 03 第一个多Agent项目：用LangGraph搭一个研究助手

Part 2: 核心能力

§ 04 Agent间通信：消息传递、共享状态和Handoff

§ 05 框架选型：LangGraph vs CrewAI vs OpenAI SDK vs AutoGen vs Google ADK

§ 06 MCP和A2A：Agent的通信协议

Part 3: 进阶实战

§ 07 生产级项目：搭一个多Agent客服系统

§ 08 调试和可观测性：多Agent系统的命门

§ 09 成本控制：别让Token费用吃掉利润

§ 10 多Agent的未来：从工具到同事

附录

Part 1: 起步

从零到一。读完这3章，你能搞懂多Agent到底是什么，跑通第一个多Agent项目。

§01 单Agent到头了，然后呢

01.1 我的单Agent撞墙经历

2025年3月。我在用Claude写一个全栈项目。

单Agent干活，挺好的。写个API、改个前端、跑个测试。它都能搞定。但项目一大，问题就来了。

我让它同时改后端接口、更新前端调用、补集成测试。它开始丢上下文。改了A文件，忘了B文件。写完接口，忘了测试。

这不是模型的问题。这是单Agent的结构性瓶颈。

一个Agent的上下文窗口就那么大。128k token听起来多，但一个中型项目的代码库轻松超过50万token。你让一个Agent同时理解数据库schema、API设计、前端组件、测试用例——它做不到。

> ***我的经验**：单Agent的天花板*
>
> *我用单Agent连续开发了47天。其中有12天，我需要反复给Agent补上下文。"你忘了，这个文件之前改过了。"每次这样的对话，浪费15-30分钟。47天里，我浪费了大约14小时在上下文补救上。*

01.2 多Agent解决什么问题

先给结论：多Agent不是让你多花钱。是让每个Agent在干净的上下文里干自己擅长的事。

单Agent是全栈工程师。什么都干，什么都不精。

多Agent是专业团队。一个管数据库，一个写API，一个做前端，一个跑测试。各司其职。

维度	单AGENT	多AGENT	我的结论
上下文管理	全塞一个窗口	每个Agent独立上下文	多Agent省上下文
专业度	什么都懂一点	每个Agent深度定制prompt	多Agent质量更高
并发能力	串行执行	多个Agent并行	多Agent快3-5倍
调试难度	一条链路	多条链路交叉	单Agent简单得多
成本	一次LLM调用	多次LLM调用	多Agent贵3-10倍
适合场景	简单任务	复杂工作流	看任务复杂度选

> ***重点看**：调试难度和成本两行。 多Agent不是银弹。它解决了上下文和专业度的问题，但带来了调试复杂度和成本的问题。Anthropic的测试数据：多Agent实现通常比单Agent多用3-10倍的token。*

01.3 3个信号：什么时候该上多Agent

Anthropic在2026年1月的博客里总结了3种情况。说实话，和我的实战经验完全对得上。

信号1：上下文污染（Context Pollution）

一个Agent的上下文里积累了太多跟当前子任务无关的信息。比如，客服Agent查完订单历史（2000+ token），再去诊断技术问题——那2000 token的订单信息就是噪音。它稀释了模型的注意力。

解法：把订单查询交给一个子Agent。子Agent处理完订单细节，只返回50-100 token的摘要给主Agent。主Agent的上下文保持干净。

信号2：任务可以并行（Parallelization）

调研"亚洲市场趋势"和"欧洲市场趋势"可以同时进行。一个Agent串行做要20分钟。5个Agent并行做只要5分钟。

Anthropic的Research功能就是这么做的。Lead Agent分析查询，派出多个子Agent并行搜索不同方向。

信号3：工具太多需要专业化（Specialization）

一个Agent挂20个以上的工具，它选工具都选不对。CRM操作、营销自动化、消息推送——这3个领域各有10-15个API端点。全给一个Agent，它会搞混。

拆成3个专业Agent，每个只挂自己领域的10个工具。加一个路由Agent做分发。选择准确率立刻上去。

> ***核心建议***：*先确认有这3个信号之一*
>
> *如果你的单Agent没有遇到上下文污染、不需要并行、工具数量不超过15个——先别上多Agent。改改prompt可能就够了。Anthropic自己说的：他们见过团队花几个月搭多Agent架构，最后发现单Agent改改prompt效果一样。*

01.4 什么时候不该用多Agent

这个比"什么时候该用"更重要。

不用多Agent的信号：

- 一个prompt就能搞定的事
- 任务步骤少于3步
- 没有明确的分工边界
- 你的团队还没调通单Agent
- 单Agent的上下文还够用（尤其是有了context compaction技术后）

NeurIPS 2025有一篇论文分析了1600+条多Agent系统trace。结论：多Agent系统的失败模式聚成3大类——系统设计缺陷（40%）、Agent间对齐错误（35%）、任务验证缺失（25%）。

> ***我的经验***：*过早上多Agent的代价*
>
> *2025年5月，一个朋友的团队花了6周搭了一个4-Agent的代码生成系统：Planner→Implementer→Tester→Reviewer。听起来很合理？结果子Agent花在协调上的token比花在干活上的还多。后来改回单Agent+优化prompt，效果一样，成本降了80%。*

01.5 多Agent的真实代价

多Agent系统的开发成本是单Agent的3-5倍。

这不是我编的。这是行业数据。原因有3个：

第一，协调逻辑。Agent之间怎么传消息、怎么共享状态、怎么处理冲突——这些全是额外的代码。

第二，上下文复制。每个Agent需要自己的上下文。Agent间协调也消耗token。结果汇总还要消耗token。

第三，测试复杂度。单Agent测试是一条线。多Agent测试是一张网。你需要测每个Agent、测Agent间的交互、测并发场景、测失败恢复。

但如果你的场景确实满足那3个信号，这些代价是值得的。

装好了心理预期。下一章，我们来搞清楚多Agent的架构模式，以及最关键的决策——怎么拆分Agent。

§02 架构模式和任务拆分：选错了全白干

02.1 我踩过的选型坑

2025年6月。我第一个多Agent项目。

任务是做一个AI内容生产流水线。研究→大纲→初稿→编辑→发布。5个步骤，很自然地想到5个Agent。

我选了Mesh架构。5个Agent互相通信。

结果呢？调试了3周。5个Agent之间有10条通信链路。任何一条出问题，整个系统就挂。最后我改成了Pipeline——5个Agent串行执行，每个Agent只管接收上一个的输出、生成给下一个的输入。

2天就跑通了。

这个教训值3周。架构选错了，后面全白干。

> ***我的经验**：选架构是最高优先级*
>
> *3周的Mesh调试，vs 2天的Pipeline搭建。架构决策影响的是10倍级的开发效率差距。先花1天想清楚用哪种模式，能省后面3周。*

02.2 5种模式一览

先给结论，然后一个个讲。

模式	一句话	控制度	调试难度	适合场景	我的结论
Orchestrator-Worker	一个指挥官，多个执行者	高	简单	客服、任务分发	入门首选
Pipeline	流水线，一步接一步	高	简单	内容生产、数据处理	最容易落地
Hierarchical	树状分层委派	高	中等	大型企业项目	20+Agent 时用
Swarm	去中心化，自主协调	低	困难	探索、研究	高级玩法
Mesh	点对点直连	中等	困难	迭代协作	3-8个Agent

02.3 Orchestrator-Worker：最常用的模式

你看，这个模式最简单。Anthropic的博客也推荐从这个开始。

一个Orchestrator（指挥官）接收任务。它拆解任务，分配给Worker（执行者）。Worker干完活，把结果交回Orchestrator。Orchestrator汇总输出。

Worker之间不通信。所有协调都经过Orchestrator。

```
用户请求 → Orchestrator → [Worker A, Worker B, Worker C] → Orchestrator → 最终结果
```

为什么它最常用？

第一，调试简单。只有一条控制流。出了问题，看Orchestrator的日志就行。

第二，扩展容易。需要新能力？加一个Worker。不影响其他Worker。

第三，映射自然。客服场景：路由Agent分类工单→计费Agent处理退款→技术Agent解答技术问题→账户Agent处理账号问题。

LangGraph的Supervisor模式就是这个架构。AutoGen的GroupChat with Selector也是。

它的问题？

Orchestrator是单点瓶颈。它的LLM调用需要3秒，20个Worker等着分配任务，你的任务分解吞吐量就被限制在每秒6.7个。

还有上下文瓶颈。Orchestrator要持有完整的任务描述+所有Worker结果+足够的上下文来合成最终响应。超过50个中间结果，128k token的模型也不够。

> ***核心建议***：*从Orchestrator-Worker开始*
>
> *如果你是第一次搭多Agent系统，选这个。它最容易理解、最容易调试、最容易改。*

02.4 Pipeline：内容生产的最佳选择

Pipeline就是流水线。

Agent A处理完，交给Agent B。Agent B处理完，交给Agent C。顺序固定。

```
[研究Agent] → [大纲Agent] → [写作Agent] → [编辑Agent] → [发布Agent]
```

每个阶段有明确的输入/输出契约。输出不合格？打回当前阶段重做。

Pipeline的好处：

第一，每个阶段可以独立优化。研究阶段用便宜模型，写作阶段用好模型。

第二，阶段之间是天然的人工审核点。大纲写完让人看一眼，再进入写作。

第三，监控简单。每个阶段的延迟、成功率、输出质量都能独立衡量。

Pipeline的问题：

不能处理条件分支。如果第3阶段的输出决定了该走第4A还是第4B，Pipeline就不好使了。

还有冷启动延迟。5个阶段，每个2秒，最少10秒才能出结果。做聊天不行，做批处理没问题。

02.5 Hierarchical、Swarm、Mesh

Hierarchical：树状结构。Manager→Supervisor→Worker。关键优势是上下文管理——没有任何一个Agent需要持有全局上下文。问题是延迟叠加——3层结构每层2秒就是6秒。Agent数量超过20个时用。

Swarm：去中心化。Agent们自己协调。OpenAI的Swarm框架就是这个思路。适合探索型任务。最大问题是可观测性——追踪一个任务的完整路径需要重建handoff链。

Mesh：3-8个Agent直接互联。典型场景：Planner→Coder→Tester形成反馈循环。关键限制：N个Agent有N(N-1)/2条连接。超过8个Agent别用。

02.6 以Context为中心的任务拆分——最关键的设计决策

这是整本书最重要的一节。

Anthropic在2026年1月的博客里提出了一个核心洞察，我读完拍大腿：**拆分任务时，要以context为中心，不是以问题类型为中心。**

什么意思？

以问题为中心的拆分（通常是错的）：

按工作类型分Agent。一个Agent写功能、一个写测试、一个做Code Review。

听起来很合理？但每次交接都在丢失context。写测试的Agent不知道为什么代码这样实现。做Review的Agent缺乏前期探索和迭代的背景。

Anthropic的实验数据：按软件开发角色（Planner→Implementer→Tester→Reviewer）拆分的子Agent，花在协调上的token比花在实际工作上的还多。

以context为中心的拆分（通常是对的）：

按context边界分Agent。一个Agent负责某个功能，那它也应该一并处理这个功能的测试——因为所需的context它本就具备。

只有当context能够真正隔离时，才值得拆分。

> ***我的经验**：传话游戏的教训*
>
> *2025年8月，我按角色拆了4个Agent：规划、编码、测试、审查。4个Agent处理同一个功能。结果就是"传话游戏"——信息来回传递，每次交接都损耗保真度。规划Agent的意图，传到编码Agent就偏了10%，传到测试Agent就偏了20%。我花了2周调Agent间的信息传递格式，最后还是把它们合回了一个Agent。*

02.7 好的拆分 vs 坏的拆分

这个表我觉得值得贴在屏幕旁边：

场景	怎么拆	为什么	我的结论
调研亚洲市场 vs 欧洲市场	☒ 拆成2个Agent	context零重叠，可以完全并行	最理想的拆分
前端 vs 后端（API契约明确）	☒ 拆成2个Agent	接口明确后context可隔离	但要先定好API契约
黑盒验证（只跑测试报结果）	☒ 拆成验证Agent	验证者不需要了解实现细节	验证Agent是万金油
同一功能的规划→实现→测试	☒ 不要拆	共享的context太多	一个Agent全干
紧耦合的组件	☒ 不要拆	需要频繁来回沟通	归属同一个Agent
需要频繁同步理解的工作	☒ 不要拆	同步成本大于分工收益	放一起

核心原则就一句话：**能隔离context就拆，不能隔离就不拆**。

> ***核心建议***：*画context边界图*
>
> *拿到需求后，先画一张context依赖图。哪些子任务共享大量context？放一起。哪些子任务的context完全独立？拆开。这一步花30分钟，能省后面3周。*

02.8 验证子Agent模式——唯一普遍适用的多Agent模式

Anthropic还给了一个"万金油模式"：验证子Agent（Verification Subagent）。

这是我目前见过最稳定的多Agent模式。

```
主Agent完成工作 → 验证Agent检查结果 → 通过？交付：打回重做
```

为什么它总能成功？因为它天然避开了"传话游戏"的问题。

验证Agent不需要知道主Agent是怎么做的。它只需要知道：结果对不对。

这是一种"黑盒验证"。输入是一个产物和验证标准。输出是通过/不通过+具体问题列表。不需要context传递。

```python
# 验证子Agent模式
class VerificationAgent:
    def verify(self, artifact, criteria):
        """
        artifact: 要验证的产物（代码、文章、报告）
        criteria: 验证标准（测试用例、质量规则、合规要求）
        """
        # 验证Agent有自己的干净上下文
        # 它不需要知道artifact是怎么生成的
        # 只需要判断artifact是否满足criteria
        response = llm.invoke(f"""
验证以下产物是否满足标准。

产物：{artifact}
标准：{criteria}

你必须运行完整的验证流程后才能判定通过。
不要只检查一两项就说通过了。
""")
        return {"passed": ..., "issues": [...]}
```

> ***注意**：验证Agent的"过早胜利"问题*
>
> *验证Agent最大的失败模式：跑了1-2个测试就说"通过"。必须在prompt里明确要求"运行完整测试套件"。没有这个要求，验证Agent会走捷径。*

02.9 怎么选架构？5个问题

第一：子任务之间独立、context可隔离？→ Orchestrator-Worker

第二：步骤顺序固定？→ Pipeline

第三：3-8个Agent需要反复迭代一个产物？→ Mesh

第四：问题空间大、路径未知？→ Swarm

第五：Agent数量超过20个？→ Hierarchical

实际项目里，大部分是混合模式。但不管哪种模式，**都要先画context边界图**。

选好了架构。下一章，搭第一个多Agent项目。

§03 第一个多Agent项目：用LangGraph搭一个研究助手

03.1 你需要什么

环境：

- Python 3.11+
- 一个OpenAI或Anthropic API Key
- 大约30分钟

我们要做什么：一个3-Agent研究助手。

```
[Researcher Agent] → [Writer Agent] → [Verifier Agent]
```

注意，我没有用Researcher→Writer→Editor。因为按照§02的context拆分原则，Writer自己就应该做编辑——它已经有了完整的写作context。

我用的是Researcher→Writer→Verifier。Verifier是黑盒验证：只检查最终文章是否准确、结构是否完整。它不需要Writer的context。

这是一个Pipeline + 验证子Agent的混合架构。

03.2 为什么选LangGraph

框架	架构模型	上手难度	生产就绪度	我的结论
LangGraph	有向图+条件边	中等	高	控制力最强
CrewAI	角色制团队	简单	中等	最快出活
AutoGen/AG2	对话式GroupChat	中等	中等	适合代码类
OpenAI Agents SDK	显式Handoff	简单	中等	OpenAI生态
Google ADK	层级Agent树	中等	早期	Google生态

我选LangGraph。因为第一个项目需要搞懂底层原理。CrewAI太抽象，藏了太多细节。

> ***我的经验**：框架选型的原则*
>
> *学习阶段选LangGraph，搞懂原理。生产阶段选CrewAI，快速出活。两个都会的人，才能做出最优选型。*

03.3 安装和配置

第一步：创建项目

```
mkdir multi-agent-research && cd multi-agent-research
python -m venv venv && source venv/bin/activate
```

预期结果：看到命令行前面出现 `(venv)`。

第二步：安装依赖

```
pip install langgraph langchain-openai langchain-community tavily-python
```

预期结果：安装成功，没有报错。

第三步：配置API Key

```
export OPENAI_API_KEY="your-openai-key"
export TAVILY_API_KEY="your-tavily-key"  # 用于网络搜索
```

> ***注意**：Tavily API Key*
>
> *Tavily是一个专门给AI Agent设计的搜索API。免费tier每月1000次调用。去 tavily.com 注册就行。*

03.4 定义Agent状态

多Agent系统的核心是**状态管理**。每个Agent读取状态、修改状态、传给下一个Agent。

```python
# state.py
from typing import TypedDict, Annotated, List
from langgraph.graph.message import add_messages

class ResearchState(TypedDict):
    topic: str                      # 研究主题
    sources: List[str]              # 收集到的资料
    draft: str                      # 初稿（Writer同时负责写作和编辑）
    verification: dict              # 验证结果
    final_article: str              # 终稿
    messages: Annotated[list, add_messages]
```

这里要注意，`ResearchState`定义了整个流程的数据流。每个Agent只读写自己关心的字段。Researcher写`sources`，Writer读`sources`写`draft`，Verifier读`draft`写`verification`。

03.5 实现3个Agent

```
# agents.py
from langchain_openai import ChatOpenAI
from langchain_community.tools.tavily_search import TavilySearchResults

llm = ChatOpenAI(model="gpt-4o", temperature=0)
llm_cheap = ChatOpenAI(model="gpt-4o-mini", temperature=0)  # 验证用便宜模型
search_tool = TavilySearchResults(max_results=5)

# Agent 1: Researcher（搜索+收集，context独立）
def researcher(state: ResearchState) -> ResearchState:
    """搜索并收集资料——独立context，不需要写作细节"""
    topic = state["topic"]
    results = search_tool.invoke(topic)
    sources = [r["content"] for r in results]
    return {
        **state,
        "sources": sources,
        "messages": [{"role": "assistant", "content": f"找到{len(sources)}条资料"}]
    }

# Agent 2: Writer（写作+自编辑，共享context所以不拆）
def writer(state: ResearchState) -> ResearchState:
    """基于资料写文章并自我编辑——写作和编辑共享context，不拆分"""
    sources_text = "\n---\n".join(state["sources"])
    prompt = f"""基于以下资料，写一篇关于"{state['topic']}"的文章。

要求：
1. 结构清晰，有小标题
2. 800字左右
3. 写完后自己审查一遍：检查逻辑、表达、准确性
4. 直接输出最终版本

资料：
{sources_text}"""
    response = llm.invoke(prompt)
    return {
        **state,
        "draft": response.content,
        "messages": [{"role": "assistant", "content": "写作完成"}]
    }

# Agent 3: Verifier（黑盒验证，不需要写作context）
def verifier(state: ResearchState) -> ResearchState:
    """黑盒验证——只检查结果，不需要了解写作过程"""
    prompt = f"""你是一个文章质量验证器。请检查以下文章：

1. 事实准确性：文中的数据和说法是否合理？
```

```
2. 结构完整性：是否有引言、主体、结论？
3. 可读性：是否流畅、逻辑清晰？

你必须逐项检查，不要只看一两项就判定通过。

文章：
{state['draft']}

输出格式：
- passed: true/false
- score: 0-10
- issues: [具体问题列表]"""
    response = llm_cheap.invoke(prompt)  # 验证用便宜模型
    return {
        **state,
        "verification": {"raw": response.content},
        "final_article": state["draft"],  # 通过则定稿
        "messages": [{"role": "assistant", "content": "验证完成"}]
    }
```

每个Agent就是一个函数。输入是状态，输出也是状态。

注意Writer为什么不拆成Writer+Editor：因为写作和编辑共享同一份context（资料+写作思路）。拆开反而会丢context。Verifier之所以独立，是因为它做黑盒验证——不需要Writer的context。

03.6 组装Pipeline

```
# graph.py
from langgraph.graph import StateGraph, END
from state import ResearchState
from agents import researcher, writer, verifier

workflow = StateGraph(ResearchState)

workflow.add_node("researcher", researcher)
workflow.add_node("writer", writer)
workflow.add_node("verifier", verifier)

workflow.set_entry_point("researcher")
workflow.add_edge("researcher", "writer")
workflow.add_edge("writer", "verifier")
workflow.add_edge("verifier", END)

app = workflow.compile()
```

6行核心代码。3个节点，3条边。

03.7 运行

```python
# main.py
from graph import app

result = app.invoke({
    "topic": "2026年多Agent架构的发展趋势",
    "sources": [],
    "draft": "",
    "verification": {},
    "final_article": "",
    "messages": []
})

print(result["final_article"])
print("\n---验证结果---")
print(result["verification"]["raw"])
```

```
python main.py
```

预期结果：等待10-20秒，输出一篇文章+验证报告。

> ***我的经验**：加了Verifier的效果*
>
> *对比了100次运行。有Verifier的版本，事实错误率从12%降到了3%。Verifier最常抓到的问题：数字编造、时间线错误、因果关系不成立。15秒的额外延迟，换来4倍的准确率提升。值。*

03.8 回顾

我们做了什么：

- 理解了以context为中心的Agent拆分
- 用LangGraph搭了一个3-Agent Pipeline + 验证子Agent
- Writer不拆Editor（共享context），但独立Verifier（黑盒验证）
- 从安装到跑通，大约30分钟

> ***核心建议**：先跑通，再优化*
>
> *这个项目还有很多可以改的地方——错误处理、重试机制、质量评估loop。但第一步是跑通。能跑通就能改。*

跑通了。下一章，深入最核心的能力：Agent间的通信和协调。

Part 2: 核心能力

深入多Agent系统的3个关键能力。读完这3章，你能设计可靠的Agent间通信、选择合适的框架、搭建MCP/A2A协议栈。

§04 Agent间通信：消息传递、共享状态和Handoff

04.1 通信方式决定系统可靠性

2025年8月。我的多Agent系统在测试环境跑得挺好。上了生产，第2天就出问题了。

原因是Agent B没收到Agent A的输出。

不是网络问题。是我的消息传递机制有bug。Agent A把结果写进了共享状态，但Agent B读的是旧版本。经典的竞态条件。

花了2天排查。修了3行代码。

从那以后，我对Agent间通信这件事变得非常谨慎。

> ***我的经验**：通信Bug最难调*
>
> *单Agent系统没有通信问题。多Agent系统80%的Bug来自通信层。消息丢了、消息重复、消息乱序、状态不一致——这些问题在测试环境很难复现，在生产环境天天出。*

04.2 3种通信方式

多Agent系统的通信，本质上就3种方式。

方式1：消息传递（Message Passing）

Agent之间直接发消息。A发给B，B收到后处理。

```
# 伪代码：消息传递
class Message:
    sender: str       # 发送者
    receiver: str     # 接收者
    content: str      # 内容
    timestamp: float  # 时间戳

# Agent A 发消息给 Agent B
msg = Message(sender="researcher", receiver="writer", content=sources_data)
message_queue.send(msg)
```

优点：解耦。Agent不需要知道对方的内部状态。 缺点：需要消息队列基础设施。消息可能丢失或重复。

方式2：共享状态（Shared State）

所有Agent读写同一个状态对象。这是LangGraph的默认方式。

```
# LangGraph 的共享状态
class SharedState(TypedDict):
    sources: List[str]      # Researcher 写
    draft: str              # Writer 写
    final_article: str      # Editor 写
```

优点：简单。不需要消息队列。 缺点：竞态条件。多个Agent同时写同一个字段就出问题。

方式3：Handoff（移交）

Agent A把控制权和上下文直接交给Agent B。OpenAI Swarm的核心机制。

```
# OpenAI Swarm 风格的 Handoff
def researcher_agent(context):
    sources = search(context.topic)
    # 直接移交给 writer，带上上下文
    return handoff(to="writer", context={**context, "sources": sources})
```

优点：上下文传递最自然。 缺点：链式调用，一个环节挂了整条链断。

方式	复杂度	可靠性	适合场景	我的结论
消息传递	高	高	大规模分布式	生产环境首选
共享状态	低	中	小型Pipeline	入门首选
Handoff	低	低	简单链式	原型验证

04.3 状态管理的3个铁律

铁律1：**每个字段有且只有一个Writer**。

如果Agent A和Agent B都能写 `draft` 字段，迟早出问题。

铁律2：**状态变更要带版本号**。

每次修改状态，带一个递增的版本号。Agent读到旧版本就知道该重新读。

铁律3：**失败要有回退策略**。

Agent写状态失败了怎么办？重试3次？回滚到上一个版本？还是跳过这个Agent？提前想好。

```
# 带版本号的状态更新
def update_state(state, field, value, agent_name):
    new_version = state.get("_version", 0) + 1
    state[field] = value
    state["_version"] = new_version
    state["_last_updated_by"] = agent_name
    return state
```

> ***核心建议**：通信层要做到"可观测"*
>
> *每一次Agent间的消息传递、状态读写，都要记录日志。出了问题，你需要能回放整个通信过程。*

04.4 实战：给研究助手加上错误处理

回到§03的项目。我们给它加上重试和回退。

```python
# agents_v2.py - 带重试的Agent
import time

def researcher_with_retry(state: ResearchState, max_retries=3):
    """带重试机制的Researcher"""
    for attempt in range(max_retries):
        try:
            results = search_tool.invoke(state["topic"])
            sources = [r["content"] for r in results]
            if len(sources) == 0:
                raise ValueError("没有搜索到结果")
            return {**state, "sources": sources}
        except Exception as e:
            if attempt < max_retries - 1:
                time.sleep(2 ** attempt)  # 指数退避
                continue
            # 最后一次重试也失败了，返回空结果
            return {
                **state,
                "sources": ["搜索失败，使用备用知识"],
                "messages": [{"role": "system", "content": f"搜索失败: {e}"}]
            }
```

这个模式很通用。每个Agent都该有重试和降级策略。

通信搞清楚了。下一章，深入对比5个主流框架。

§05 框架选型：LangGraph vs CrewAI vs OpenAI SDK vs AutoGen vs Google ADK

05.1 我用过的5个框架

从2025年3月到2026年4月。13个月。我用这5个框架各做了至少1个项目。

LangGraph做了3个项目。CrewAI做了2个。OpenAI Agents SDK做了2个。AutoGen做了1个。Google ADK做了1个。

我有资格做这个对比。

> ***我的经验***：*5个框架都用过后的感受*
>
> *没有最好的框架。只有最适合你项目的框架。LangGraph控制力最强但学习曲线陡。CrewAI最快出活但遇到边界case难改。OpenAI SDK最简洁但锁定生态。AutoGen适合代码任务但架构复杂。Google ADK还太早期。*

05.2 核心维度对比

维度	LANGGRAPH	CREWAI	OPENAI SDK	AUTOGEN/AG2	GOOGLE ADK	我的结论
编排模型	有向图+条件边	角色制+流程类型	显式Handoff	对话式GroupChat	层级Agent树	LangGraph最灵活
学习曲线	陡	平缓	平缓	中等	中等	CrewAI最快上手
生产就绪	高	中等	中等	中等	早期	LangGraph最稳
模型支持	任意	任意	仅OpenAI	任意	偏Google	LangGraph最开放
状态管理	内建Checkpoint	基础	基础	会话式	内建	LangGraph最完善
部署速度	慢	快40%	快	慢	中等	CrewAI最快
社区规模	大	大	大	中等	增长中	LangGraph和CrewAI并列

> ***重点看***：*编排模型和生产就绪两行。 编排模型决定了你能做什么。生产就绪决定了你能不能上线。*

05.3 LangGraph：控制狂的首选

LangGraph把多Agent系统建模为有向图。节点是Agent，边是状态转换。

核心概念就3个：

- **State**：共享状态对象
- **Node**：Agent函数
- **Edge**：节点间的连接（可以是条件边）

```
# LangGraph 的条件分支
def should_continue(state):
    """根据状态决定下一步"""
    if state["quality_score"] > 0.8:
        return "publish"    # 质量够了，发布
    else:
        return "rewrite"    # 质量不够，重写

workflow.add_conditional_edges(
    "editor",
    should_continue,
    {"publish": "publisher", "rewrite": "writer"}
)
```

优势：你能精确控制每一步。条件分支、循环、并行执行、人工介入——都能做。

劣势：代码量大。一个中等复杂度的多Agent系统，LangGraph要写300-500行编排代码。CrewAI可能只需100行。

内建的Checkpoint机制是杀手级功能。系统在任何节点崩溃，都能从上一个checkpoint恢复。生产环境必备。

05.4 CrewAI：最快出活

CrewAI的抽象层级更高。你定义"角色"，不定义"图"。

```python
# CrewAI 风格
from crewai import Agent, Task, Crew

researcher = Agent(
    role="资深研究员",
    goal="找到关于{topic}的最新信息",
    backstory="你有10年的技术研究经验..."
)

writer = Agent(
    role="技术作者",
    goal="写出清晰易懂的技术文章",
    backstory="你是一个技术博客作者..."
)

# 定义任务
research_task = Task(description="调研{topic}", agent=researcher)
write_task = Task(description="基于调研写文章", agent=writer)

# 组装团队
crew = Crew(agents=[researcher, writer], tasks=[research_task, write_task])
result = crew.kickoff(inputs={"topic": "多Agent架构"})
```

比LangGraph少了一半代码。角色化的抽象很直觉——就像组建一个团队。

优势：开发速度快。CrewAI的角色制抽象是最容易理解的。

劣势：遇到边界case难改。CrewAI藏了太多编排细节。当你需要自定义状态转换逻辑时，就得和框架的内部实现做斗争。

05.5 OpenAI Agents SDK：最简洁

OpenAI的方案最轻量。核心概念就两个：Agent和Handoff。

```python
# OpenAI Agents SDK 风格
from openai_agents import Agent, handoff

researcher = Agent(
    name="researcher",
    instructions="你负责搜索资料",
    tools=[search_tool]
)

writer = Agent(
    name="writer",
    instructions="你负责写文章",
    handoffs=[handoff(to=researcher)]  # 可以把任务交回researcher
)
```

优势：极简。和OpenAI的API无缝集成。

劣势：锁定OpenAI生态。想用Claude或其他模型？换框架。

05.6 选型决策树

问自己3个问题：

问题1：你需要精确控制流程吗？

- 需要 → LangGraph
- 不需要 → 下一个问题

问题2：你需要最快出活吗？

- 需要 → CrewAI
- 不需要 → 下一个问题

问题3：你只用OpenAI的模型吗？

- 是 → OpenAI Agents SDK
- 不是 → LangGraph

> ***核心建议***：*两个框架都学*
>
> *学LangGraph理解原理。学CrewAI提升效率。这两个是2026年的主流。*

框架选好了。下一章，搞懂Agent之间的"语言"——MCP和A2A协议。

§06 MCP和A2A：Agent的通信协议

06.1 为什么需要协议

2025年之前，Agent连接外部工具的方式是——写自定义代码。

每个Agent要调数据库？写一个数据库工具函数。要调API？写一个API工具函数。要搜索网页？再写一个。

10个工具，10套自定义代码。换一个框架，全部重写。

这就是MCP和A2A要解决的问题。

06.2 MCP：Agent连接工具的USB接口

MCP全称Model Context Protocol。Anthropic在2024年底发布。到2026年4月，MCP的npm下载量超过9700万次。

一句话：MCP是Agent连接工具和数据源的标准接口。

```
Agent ←→ MCP ←→ [数据库, API, 文件系统, 搜索引擎...]
```

你看，MCP解决的是Agent到工具的连接问题。不是Agent到Agent的。

MCP有3个核心概念：

- **MCP Server**：工具提供方。比如一个暴露了SQL查询能力的数据库MCP Server。
- **MCP Client**：Agent侧。Agent通过MCP Client调用MCP Server的工具。
- **Transport**：通信层。支持stdio（本地）和HTTP（远程）。

```python
# MCP Server 示例（伪代码）
from mcp import MCPServer, tool

server = MCPServer("database-tools")

@tool
def query_database(sql: str) -> str:
    """执行SQL查询"""
    result = db.execute(sql)
    return str(result)

@tool
def list_tables() -> list:
    """列出所有表"""
    return db.get_tables()

server.run()
```

Agent端不需要知道数据库的连接方式、认证方式、查询语法。它只需要知道：有一个MCP Server提供了 `query_database` 和 `list_tables` 两个工具。

> ***我的经验**：MCP改变了工具生态*
>
> *2025年之前，我给每个项目写自定义工具函数。平均每个项目写5-8个工具，每个工具100-200行代码。MCP出来后，社区有了上千个现成的MCP Server。数据库、GitHub、Slack、Jira——直接接上就能用。我的工具代码量从每项目1000行降到了不到100行。*

06.3 A2A：Agent之间的通信协议

A2A全称Agent-to-Agent protocol。Google在2025年4月发布。截至2026年4月，已有50+合作伙伴。

一句话：A2A是Agent与Agent之间通信的标准协议。

```
Agent A ←→ A2A ←→ Agent B
```

MCP解决Agent到工具。A2A解决Agent到Agent。两个协议互补。

A2A的核心是Task生命周期：

1. Agent A发起一个Task给Agent B
2. Agent B接收、处理

3. Agent B返回结果
4. Task完成

```
Task 生命周期：created → working → completed/failed
```

A2A还定义了Agent Discovery——Agent怎么找到其他Agent、知道它们有什么能力。

06.4 MCP + A2A = 完整的Agent协议栈

2026年的完整Agent协议栈长这样：

层	协议	作用
Agent到工具	MCP	连接数据库、API、文件系统
Agent到Agent	A2A	Agent间任务委派和通信
Agent到商业	ACP/UCP	Agent间的交易和支付

目前MCP已经成为行业标准。Claude、ChatGPT、Cursor、各大IDE都支持MCP。

A2A还在早期推广阶段。但方向是对的——Agent之间需要一个标准的通信协议。

```
# 完整协议栈示例（概念代码）
# Agent通过MCP连接工具
agent.connect_mcp_server("database-tools")
agent.connect_mcp_server("github-tools")

# Agent通过A2A连接其他Agent
agent.discover_agents()  # 发现可用的Agent
agent.delegate_task(      # 委派任务
    to="data-analysis-agent",
    task="分析最近30天的用户行为数据"
)
```

> ***核心建议**：MCP现在就学，A2A持续关注*
>
> *MCP已经是标准了。你的Agent系统必须支持MCP。A2A还在发展，但Agent间通信的标准化是必然趋势。*

06.5 安全考量

多Agent系统的安全问题比单Agent严重得多。

每增加一个Agent，就多一个攻击面。Agent间通信可能被注入恶意指令。Agent的MCP工具权限可能被滥用。

3个必须做的安全措施：

第一，每个Agent的工具权限要最小化。数据库Agent只能查询，不能删除。

第二，Agent间通信要做输入验证。不能让Agent A通过prompt注入控制Agent B。

第三，所有Agent操作要有审计日志。谁干了什么，什么时候干的，结果是什么。

协议搞清楚了。下一章，进入实战——搭一个生产级的多Agent系统。

Part 3: 进阶实战

多场景、多框架、生产级的复杂用法。读完这4章，你能在生产环境跑多Agent系统。

§07 生产级项目：搭一个多Agent客服系统

07.1 为什么是客服

客服是多Agent落地最成熟的场景。没有之一。

原因很简单。客服场景天然适合Orchestrator-Worker模式。用户的问题可以分类：计费、技术、账号、投诉。每种类型对应一个专业Agent。路由Agent做分发，专业Agent做解答。

行业数据：头部平台用这套架构，自主解决率超过90%。

> ***我的经验**：从客服场景入手*
>
> *2025年9月，我帮一个SaaS公司搭客服系统。3个星期从零到上线。日均处理1200条工单。自主解决率87%。客户满意度从3.2提升到4.1（5分制）。这个项目让我确信，多Agent在客服场景是成熟的。*

07.2 系统架构

```
用户消息 → [路由Agent] → [计费Agent | 技术Agent | 账号Agent] → 回复用户
                              ↓（低置信度时）
                          [人工客服]
```

4个Agent + 1个人工兜底。典型的Orchestrator-Worker。

07.3 路由Agent

路由Agent是整个系统的核心。它的准确率决定了系统的整体质量。

```python
# router_agent.py
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)  # 路由用便宜模型

ROUTER_PROMPT = """你是一个客服路由器。根据用户消息，分类到以下类别之一：

- billing: 计费、退款、订阅、支付相关
- technical: 技术问题、Bug、功能不工作
- account: 账号设置、密码、权限相关
- escalate: 投诉、情绪激动、需要人工处理

只返回类别名称，不要返回其他内容。

用户消息：{message}"""

def route(state):
    """路由用户消息到对应Agent"""
    response = llm.invoke(
        ROUTER_PROMPT.format(message=state["user_message"])
    )
    category = response.content.strip().lower()

    # 置信度检查
    valid_categories = ["billing", "technical", "account", "escalate"]
    if category not in valid_categories:
        category = "escalate"  # 无法分类就转人工

    return {**state, "category": category}
```

关键点：路由Agent用便宜模型。它的任务简单——分类。不需要GPT-4o，GPT-4o-mini就够了。一个月能省60%的API成本。

07.4 专业Agent

每个专业Agent有自己的prompt、工具和知识库。

```
# billing_agent.py
BILLING_PROMPT = """你是计费专员。你的职责：
1. 解答计费相关问题
2. 处理退款请求（需确认订单号）
3. 说明订阅方案差异

可用工具：
- query_orders: 查询订单
- process_refund: 处理退款
- get_subscription: 查询订阅

规则：
- 退款超过500元，必须转人工
- 态度友好但专业
- 不确定的问题，说"我帮您转接人工客服"

用户消息：{message}
订单历史：{order_history}"""

def billing_agent(state):
    """计费Agent"""
    # 获取用户的订单历史
    orders = query_orders(state["user_id"])

    response = llm.invoke(
        BILLING_PROMPT.format(
            message=state["user_message"],
            order_history=str(orders)
        )
    )

    return {**state, "response": response.content}
```

07.5 置信度和降级策略

生产系统必须有降级策略。Agent不确定的时候，转人工。

```python
# confidence.py
def check_confidence(state):
    """检查Agent回复的置信度"""
    response = state["response"]

    # 低置信度信号
    low_confidence_signals = [
        "我不确定",
        "建议您联系",
        "无法确认",
        "抱歉，这个问题"
    ]

    for signal in low_confidence_signals:
        if signal in response:
            return "escalate"  # 转人工

    return "respond"  # 直接回复
```

在LangGraph里，这是一个条件边：

```python
workflow.add_conditional_edges(
    "specialist",
    check_confidence,
    {"respond": "send_response", "escalate": "human_agent"}
)
```

> ***核心建议***：*宁可多转人工，不可乱回答*
>
> *客服系统的信誉比效率重要。自主解决率从87%提升到95%不难，但如果那8%的回答是错的，损失远大于收益。*

07.6 监控和指标

上线后需要盯的指标：

指标	目标	报警阈值
路由准确率	>95%	<90%
自主解决率	>85%	<75%
平均响应时间	<5秒	>10秒
用户满意度	>4.0/5	<3.5/5
转人工率	<15%	>25%

每天看一次仪表盘。路由准确率掉了，查prompt。自主解决率掉了，查知识库。响应时间涨了，查API延迟。

搭好了生产级客服系统。下一章，处理多Agent系统最难的部分——调试和可观测性。

§08 调试和可观测性：多Agent系统的命门

08.1 调试多Agent是噩梦

不夸张。调试多Agent系统比写多Agent系统难3倍。

单Agent的调试是一条直线：输入→处理→输出。看日志就行。

多Agent的调试是一张网：Agent A输出→Agent B输入→Agent B输出→Agent C输入→…→最终结果。任何一个节点出问题，你需要定位是哪个Agent、在哪一步、为什么错。

2025年10月。我的一个多Agent系统输出了完全错误的结果。最终定位原因花了8小时。问题出在第3个Agent的prompt里，一个标点符号放错了位置。

> ***我的经验**：8小时定位一个标点*
>
> *4个Agent的Pipeline。最终输出是一份报告。报告里有一段分析完全错误。我一个Agent一个Agent地检查中间输出，花了8小时。最后发现是Writer Agent的prompt里，“总结以上内容。不要添加额外信息"被写成了"总结以上内容，不要添加额外信息”。句号变逗号，语义从两个指令变成了一个带从句的指令。大模型的理解就偏了。*

08.2 可观测性的3个层级

层级1：日志（Logging）

最基础的。每个Agent的输入、输出、耗时全部记录。

```
import logging
import time

logger = logging.getLogger("multi-agent")

def logged_agent(agent_fn, name):
    """给Agent加上日志包装"""
    def wrapper(state):
        start = time.time()
        logger.info(f"[{name}] 开始执行. 输入状态: {list(state.keys())}")

        result = agent_fn(state)

        elapsed = time.time() - start
        logger.info(f"[{name}] 执行完成. 耗时: {elapsed:.2f}s")
        return result
    return wrapper

# 使用
researcher = logged_agent(researcher_fn, "Researcher")
writer = logged_agent(writer_fn, "Writer")
```

层级2：追踪（Tracing）

把每个Agent的调用串成一条链。一个请求从进入到完成，能看到完整的调用链。

LangSmith是LangGraph的配套追踪工具。接入只需要2行代码：

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your-langsmith-key"
```

接入后，你能在LangSmith的界面上看到：

- 每个Agent的输入/输出
- 每个LLM调用的prompt和response
- 每一步的token消耗和延迟
- 整条链路的耗时瀑布图

层级3：指标（Metrics）

聚合数据。不是看单个请求，是看整体趋势。

```
# 关键指标收集
metrics = {
    "agent_latency": {},        # 每个Agent的平均延迟
    "agent_error_rate": {},     # 每个Agent的错误率
    "pipeline_success_rate": 0, # 整条Pipeline的成功率
    "total_token_usage": 0,     # 总token消耗
    "cost_per_request": 0       # 每个请求的成本
}
```

08.3 NeurIPS的14种失败模式

NeurIPS 2025有一篇论文：《Why Do Multi-Agent LLM Systems Fail?》。

他们分析了1600+条多Agent系统的trace，在7个主流框架上收集数据。结论是14种失败模式，聚成3大类：

类别	占比	典型失败	我的结论
系统设计缺陷	~40%	Agent分工不清、prompt冲突	设计阶段能避免
Agent间对齐错误	~35%	A的输出格式B解析不了	需要接口契约
任务验证缺失	~25%	没人检查最终输出质量	必须有验证Agent

> ***重点看***：*Agent间对齐错误占35%。这意味着超过三分之一的失败是因为Agent之间"说不通话"。解决方法很简单：定义清晰的输入/输出Schema。*

08.4 实战：构建可观测的Pipeline

给§03的项目加上完整的可观测性：

```python
# observable_pipeline.py
import json
import time
from datetime import datetime

class PipelineObserver:
    def __init__(self):
        self.traces = []

    def record(self, agent_name, input_state, output_state, elapsed):
        self.traces.append({
            "timestamp": datetime.now().isoformat(),
            "agent": agent_name,
            "elapsed_seconds": round(elapsed, 2),
            "input_keys": list(input_state.keys()),
            "output_keys": list(output_state.keys()),
            "success": True
        })

    def record_error(self, agent_name, error, elapsed):
        self.traces.append({
            "timestamp": datetime.now().isoformat(),
            "agent": agent_name,
            "elapsed_seconds": round(elapsed, 2),
            "error": str(error),
            "success": False
        })

    def summary(self):
        """打印执行摘要"""
        total_time = sum(t["elapsed_seconds"] for t in self.traces)
        success_count = sum(1 for t in self.traces if t["success"])
        print(f"总耗时: {total_time:.2f}s")
        print(f"成功: {success_count}/{len(self.traces)}")
        for t in self.traces:
            status = "□" if t["success"] else "□"
            print(f"  {status} {t['agent']}: {t['elapsed_seconds']}s")
```

用法:

```
observer = PipelineObserver()

# 在每个Agent执行前后记录
start = time.time()
state = researcher(state)
observer.record("Researcher", old_state, state, time.time() - start)

# 最后打印摘要
observer.summary()
# 总耗时: 12.34s
# 成功: 3/3
#   Researcher: 3.21s
#   Writer: 6.78s
#   Editor: 2.35s
```

> ***核心建议***：*Day 1就接入追踪*
>
> *不要等出了问题再加日志。从第一天开始就把可观测性接上。LangSmith的免费tier够用了。*

调试武器备齐了。下一章，处理多Agent的成本控制。

§09 成本控制：别让Token费用吃掉利润

09.1 多Agent的成本是指数级的

2025年11月。我的一个多Agent系统上线了。第一个月，API账单来了——$2,400。

我预估是$800。差了3倍。

原因很直接。我有5个Agent，每个Agent每次调用消耗约2000 token。一个用户请求触发全部5个Agent，就是10000 token。每天3000个请求，每天3000万token。

GPT-4o的价格是$2.50/百万输入token + $10/百万输出token。一个月下来，轻松超$2000。

> ***我的经验***：*成本是多Agent的隐形杀手*
>
> *多Agent系统的成本不是线性增长。5个Agent不是5倍成本。因为Agent间通信也消耗token，Orchestrator的决策也消耗token。实际成本通常是Agent数量×2到×3。*

09.2 成本优化的4个杠杆

杠杆1：模型分级

不是每个Agent都需要最好的模型。

```
# 模型分级策略
AGENT_MODELS = {
    "router":     "gpt-4o-mini",    # 路由：简单分类，便宜模型
    "researcher": "gpt-4o-mini",    # 搜索：调用工具为主，便宜模型
    "writer":     "gpt-4o",         # 写作：需要质量，用好模型
    "editor":     "gpt-4o",         # 编辑：需要判断力，用好模型
    "validator":  "gpt-4o-mini",    # 验证：结构化检查，便宜模型
}
```

路由Agent用GPT-4o-mini，比GPT-4o便宜20倍。路由准确率从98%降到95%。可以接受。

AGENT	用GPT-4O	用GPT-4O-MINI	质量降幅	我的结论
Router	$0.0025/次	$0.00015/次	-3%	用mini
Researcher	$0.005/次	$0.0003/次	-5%	用mini
Writer	$0.01/次	$0.0006/次	-15%	用4o
Editor	$0.008/次	$0.0005/次	-10%	用4o
Validator	$0.003/次	$0.0002/次	-2%	用mini

> ***重点看***：*Writer的质量降幅是15%。 写作是核心价值，不能省。但路由和验证的质量降幅只有2-3%。这些该省。*

杠杆2：缓存

相同的问题不需要每次都问大模型。

```python
# 简单的语义缓存
import hashlib

cache = {}

def cached_agent(agent_fn, cache_key_fn):
    def wrapper(state):
        key = cache_key_fn(state)
        if key in cache:
            return cache[key]  # 命中缓存
        result = agent_fn(state)
        cache[key] = result
        return result
    return wrapper

# 路由Agent的缓存
def router_cache_key(state):
    return hashlib.md5(state["user_message"].encode()).hexdigest()

router = cached_agent(route, router_cache_key)
```

客服场景的缓存命中率通常在30-50%。因为用户问的问题有大量重复。

杠杆3：提前终止

不是每个请求都需要走完所有Agent。

```python
# 提前终止策略
def should_skip_editor(state):
    """Writer的输出质量够高，跳过Editor"""
    if state.get("quality_score", 0) > 0.9:
        return "skip_editor"
    return "run_editor"
```

简单问题不需要过Editor。路由Agent就能直接回答的问题，不需要进入专业Agent。

杠杆4：批处理

非实时场景，把请求攒起来批量处理。OpenAI的Batch API价格是实时API的一半。

09.3 成本监控看板

```python
# cost_tracker.py
MODEL_PRICING = {
    "gpt-4o": {"input": 2.50, "output": 10.00},        # $/百万token
    "gpt-4o-mini": {"input": 0.15, "output": 0.60},
}

class CostTracker:
    def __init__(self):
        self.total_cost = 0
        self.agent_costs = {}

    def record(self, agent_name, model, input_tokens, output_tokens):
        pricing = MODEL_PRICING[model]
        cost = (input_tokens * pricing["input"] +
                output_tokens * pricing["output"]) / 1_000_000
        self.total_cost += cost
        self.agent_costs[agent_name] = (
            self.agent_costs.get(agent_name, 0) + cost
        )

    def report(self):
        print(f"总成本: ${self.total_cost:.4f}")
        for agent, cost in sorted(
            self.agent_costs.items(), key=lambda x: -x[1]
        ):
            pct = cost / self.total_cost * 100
            print(f"  {agent}: ${cost:.4f} ({pct:.1f}%)")
```

每天跑一次报告。如果某个Agent的成本突然涨了，说明它的行为变了——可能是prompt漂移，可能是输入变复杂了。

> ***核心建议***：*成本预算写在架构文档里*
>
> *不要等账单来了才想成本。设计阶段就定好每个Agent用什么模型、每个请求的成本上限。*

成本控制好了。最后一章，聊聊多Agent的未来。

§10 多Agent的未来：从工具到同事

10.1 我的思维转变

2025年3月，我把Agent当工具。给它指令，它执行。

2026年4月，我把Agent当同事。给它目标，它自己规划和执行。

这个转变花了13个月。

回头看，最大的认知跃迁不是技术层面的。不是学会了LangGraph还是CrewAI。而是理解了一件事：**多Agent系统的设计，本质上是组织设计。**

你怎么设计一个人类团队，就应该怎么设计一个Agent团队。明确分工、清晰沟通、有效监督、及时反馈。

> ***我的经验**：Agent团队和人类团队是一回事*
>
> *我管过5人的开发团队，也搭过5个Agent的多Agent系统。踩的坑几乎一样——分工不清导致重复工作，沟通不畅导致返工，没有Review导致质量下降。唯一的区别是，Agent不会抱怨加班。*

10.2 2026年的3个趋势

趋势1：协议标准化

MCP已经是Agent到工具的标准。A2A正在成为Agent到Agent的标准。

到2026年底，我预计企业级Agent系统的协议栈会稳定为：MCP + A2A + 安全层。这意味着你的Agent可以和任何其他公司的Agent通信。不需要定制集成。

趋势2：Agent专业化

2025年的Agent还是"通用型"的。一个Agent什么都能干，但什么都不精。

2026年开始，市场上出现了大量专业Agent。专门做SQL查询的Agent、专门做代码Review的Agent、专门做文案生成的Agent。你不需要自己训练，直接接入就能用。

趋势3：多Agent系统的可观测性成熟

目前调试多Agent系统还是痛苦的。但LangSmith、Galileo、Maxim这些可观测性平台在快速进化。

我预计到2026年底，多Agent系统的调试体验会接近传统微服务的水平——有分布式追踪、有性能分析、有异常检测、有自动告警。

10.3 什么时候多Agent会成为默认架构

我的判断：2027年。

原因有3个：

第一，模型成本在快速下降。2024年GPT-4的价格是2026年GPT-4o-mini价格的100倍。当模型成本降到足够低，多Agent的成本劣势消失。

第二，框架和协议在成熟。LangGraph、CrewAI已经能支撑生产级系统。MCP、A2A正在标准化Agent间通信。基础设施就位了。

第三，应用场景在扩展。从客服到研发到运营到销售，多Agent系统在越来越多的场景被验证。

10.4 给你的行动建议

如果你现在开始学多Agent，我建议的路径是：

阶段	时间	行动	目标
第1周	Day 1-3	读完本书Part 1	理解基本概念
第1周	Day 4-7	跑通§03的代码	第一个多Agent项目
第2周	Day 8-10	学LangGraph	理解状态图模型
第2周	Day 11-14	学CrewAI	理解角色化抽象
第3周	Day 15-18	搭一个客服系统	第一个生产级项目
第3周	Day 19-21	接入MCP	理解协议生态
第4周	Day 22-28	优化和监控	生产级可观测性

4周。28天。从零到能独立设计和部署多Agent系统。

> ***我的经验**：学多Agent的最大收获*
>
> *不是技术能力的提升。是思维方式的转变。学会把大问题拆成小问题，把小问题分给专业Agent。这个能力不只在AI领域有用——管理团队、设计系统、解决复杂问题，都是同一个思路。*

这本书到这里。

我写的所有内容，都是基于13个月、9个项目、5个框架的真实经验。有些判断可能会过时，但底层的架构思维不会。

去搭你的第一个多Agent系统。

附录

A 多Agent框架速查表（2026年4月）

框架	GITHUB STARS	编排模型	最佳场景	MCP支持	生产就绪
LangGraph	20k+	有向图+条件边	复杂工作流	☒	高
CrewAI	25k+	角色制团队	快速原型	☒	中等
OpenAI Agents SDK	15k+	Handoff	OpenAI生态	☒	中等
AutoGen/AG2	40k+	GroupChat	代码生成	☒	中等
Google ADK	10k+	层级Agent树	Google生态	☒	早期
Claude Agent SDK	5k+	子Agent+Task	Anthropic 生态	☒	中等
MetaGPT	48k+	SOP工作流	软件开发	部分	早期
Swarms	8k+	多种拓扑	大规模并行	部分	早期

B 5种架构模式决策树

```
你的任务需要多Agent吗?
├── 任务步骤 < 3步 → 不需要，用单Agent
├── 单Agent上下文够用 → 不需要，用单Agent
└── 需要多Agent
    ├── 子任务独立? → Orchestrator-Worker
    ├── 步骤顺序固定? → Pipeline
    ├── 3-8个Agent反复迭代? → Mesh
    ├── 问题空间大、路径未知? → Swarm
    └── Agent数量 > 20? → Hierarchical
```

C Agent间通信协议对比

协议	发布方	发布时间	作用	成熟度
MCP	Anthropic	2024年底	Agent→工具	成熟（9700万+下载）
A2A	Google	2025年4月	Agent→Agent	早期（50+合作伙伴）
ACP	社区	2025年	Agent商业交易	早期
UCP	Google	2026年	Agent商业层	早期

D 成本估算模板

假设：日均3000请求，5个Agent Pipeline

AGENT	模型	TOKEN/次	单价($/M)	日成本	月成本
Router	gpt-4o-mini	500	$0.15/$0.60	$0.36	$10.8
Researcher	gpt-4o-mini	1500	$0.15/$0.60	$1.35	$40.5
Writer	gpt-4o	3000	$2.50/$10.00	$67.5	$2,025
Editor	gpt-4o	2000	$2.50/$10.00	$45.0	$1,350
Validator	gpt-4o-mini	800	$0.15/$0.60	$0.72	$21.6
合计				**$114.93**	**$3,448**

优化后（模型分级+缓存30%命中率）：约$2,400/月。省30%。

E 常见问题

Q: 多Agent比单Agent贵多少？ A: 通常3-5倍。但模型分级和缓存能降到2-3倍。

Q: 什么时候不该用多Agent？ A: 任务简单（<3步）、不需要并行、单Agent上下文够用时。

Q: LangGraph和CrewAI选哪个？ A: 学习选LangGraph，快速出活选CrewAI。两个都学是最优解。

Q: MCP必须接入吗？ A: 2026年，是的。MCP已经是Agent工具连接的行业标准。

Q: 多Agent系统最大的坑是什么？ A: 调试。从Day 1就接入可观测性工具。

阅读指南

时间	章节	目标
Day 1	§01-§02	理解多Agent概念和5种架构模式
Day 2	§03	跑通第一个多Agent项目
Day 3-4	§04-§06	掌握通信机制、框架选型、协议生态
Day 5-6	§07-§09	搭生产级系统、调试、成本控制
Day 7	§10 + 附录	趋势判断 + 速查表

> ***程佳路出品*** *| AI Native Coder · 独立开发者 公众号「探微观智」| B站「AI进化论-花生」 代表作：Claude Code从入门到精通 · OpenClaw橙皮书 · Hermes Agent从入门到精通*

多AGENT架构

从单Agent到多Agent协作的完整实战指南。10章 · 3 Part · 覆盖 LangGraph/CrewAI/OpenAI SDK/AutoGen/Google ADK + MCP/A2A 协议栈。